REDIS — ARCHITECTURE, OPERATIONS & REFERENCE V1

The in-memory data structure store — what it guarantees, what it does not, and how it fails
threading · data structures · persistence · replication · cluster · locks · operations · **depth marked** junior → staff

1 · WHAT REDIS IS — THE THREADING MODEL, AND THE THREE THINGS THAT FOLLOW FROM IT

start here — the mental model that explains every performance characteristic and every failure mode on this sheet

The definition

Redis is an **in-memory data structure store.** Not a key-value cache with extras — the values are genuinely **lists, sets, sorted sets, hashes, streams and bitmaps**, each with operations that run inside the server. That is the actual differentiator.

Why it is fast, in one sentence. Data is in memory, the data structures are chosen per operation, and **commands execute one at a time on a single thread** — so there are no locks, no contention and no coordination overhead anywhere in the hot path.

The threading model, precisely

What	Threaded?
Command execution	**Single-threaded.** Commands run sequentially, one at a time, and this has not changed
Network I/O	Can be threaded since **Redis 6.0** via io-threads — reading from and writing to sockets moves off the main thread
Background work	Always used threads and forks: BGSAVE forks a child, AOF rewrite forks, UNLINK frees memory in a background thread

So "Redis is single-threaded" is a half-truth worth stating precisely. I/O parallelism exists; **command execution parallelism does not.** Even with I/O threads enabled, two commands arriving simultaneously are queued and executed in sequence.

The three consequences — everything else derives from these

1	**Every command is atomic, for free.** No transaction is needed to increment a counter or pop from a list — nothing can interleave. This is why Redis is the natural home for counters, rate limiters and locks
2	**One slow command blocks everything.** There is no second thread to serve other clients. A KEYS * on a large database, a large SORT, or deleting a million-element set stalls *every* connected client for the duration
3	**One core is the throughput ceiling per instance.** Vertical scaling stops at a single core's command rate — which is why scaling out means **more instances**, not a bigger machine

Consequence 2 is the source of most production incidents on this sheet. Command complexity is not a performance detail in Redis; it is an availability property. An O(n) command on a large collection is a self-inflicted outage, and the server will happily run it.

Where it sits

Redis is not a relational database, a queue broker, or an analytics engine — though it can be pressed into all three. It is at its best when the **working set fits in memory** and the access pattern is a data-structure operation rather than a query.

Use it when

- **Sub-millisecond latency** is a genuine requirement
- The **working set fits in memory** — this is the hard constraint
- The access pattern **maps to a data structure**: leaderboard, queue, set membership, counter, rate limit
- Caching, sessions, distributed locks, real-time counters, pub/sub, stream processing

Do not use it when

- **Data exceeds memory** and cannot be partitioned — memory is the cost driver and the limit
- You need **joins, ad-hoc queries or secondary indexes** across attributes
- **Zero data loss is required** — see the replication panel; even AOF has a window
- **Analytics over large datasets** — a columnar engine is the right tool
- You need **multi-key ACID with rollback** — Redis transactions do not roll back

STAFF **A licensing note worth knowing.** Redis moved from BSD to source-available licences (RSALv2 / SSPLv1) in 2024, then **added AGPLv3 as an OSI-approved option with Redis 8** in May 2025 — it is now tri-licensed. **Valkey** was forked from Redis in March 2024 under the Linux Foundation and remains BSD-3-Clause. Both are actively developed; the architecture on this sheet applies to both.

2 · DATA STRUCTURES — AND THE COMPLEXITY THAT DECIDES WHETHER THEY ARE SAFE

JUNIOR what each is for · MID the complexity column, which is the one that matters in production

Type	Use for	Key commands · complexity
String	Cache values, counters, flags, serialised objects	SET GET INCR SETEX GETRANGE — **O(1)**. Holds up to 512 MB, though large values are a bad idea
Hash	Objects with fields — a user profile, a session	HSET HGET HGETALL HINCRBY — O(1) per field; **HGETALL is O(n)** in field count
List	Queues, stacks, recent-items, timelines	LPUSH RPOP LRANGE LTRIM BLPOP — push and pop **O(1)**; LRANGE is O(n)
Set	Unique membership, tags, deduplication	SADD SISMEMBER SINTER SCARD — add and test **O(1)**; **SMEMBERS and set operations are O(n)**
Sorted set	Leaderboards, priority queues, rate limiting, time-ordered indexes	ZADD ZRANGE ZRANGEBYSCORE ZINCRBY — **O(log n)** add and rank. The most versatile structure here
Bitmap	Dense boolean state — daily actives, feature flags per user id	SETBIT GETBIT BITCOUNT BITOP — O(1) per bit; extremely compact
HyperLogLog	Approximate distinct counts at fixed memory	PFADD PFCOUNT PFMERGE — **approximate**, with a small standard error, in ~12 KB regardless of cardinality
Geospatial	Proximity search — built on sorted sets	GEOADD GEOSEARCH — prefer GEOSEARCH; GEORADIUS is deprecated
Stream	Append-only log with consumer groups	XADD XREADGROUP XACK XAUTOCLAIM — see panel 13

Read the complexity column as an availability column. Because execution is single-threaded, an O(n) command on a large collection blocks every other client for its whole duration. SMEMBERS on a ten-million-member set is not slow — it is an outage.

The commands to ban outright in production

Never run	Use instead
KEYS *	SCAN — cursor-based, incremental, does not block
FLUSHALL / FLUSHDB	Nothing. Rename or disable via rename-command
SMEMBERS on a big set	SSCAN
HGETALL on a big hash	HSCAN, or fetch only the fields you need
DEL on a huge collection	**UNLINK** — frees memory in a background thread rather than inline

The SCAN guarantee, stated honestly. It guarantees every element present for the whole iteration is returned at least once — but elements may be **returned more than once**, and those added or removed mid-scan may or may not appear. It is a cursor, not a snapshot, and callers must tolerate duplicates.

STAFF **Modules extend the type system.** RedisJSON adds native JSON with path queries; RediSearch adds secondary indexing and full-text; RedisTimeSeries adds downsampling and retention; RedisBloom adds Bloom and Cuckoo filters, Count-Min Sketch and Top-K. They change what Redis is suitable for — and they are a deployment dependency, so confirm availability on your managed platform first.

3 · MEMORY & ENCODINGS

what you actually pay for

Memory is the cost and the limit. Redis stores far more than the raw bytes of your data — every key carries overhead for the key string, the value object, the expiry, and hash-table slots.

The rule of thumb worth internalising. Many small keys are dramatically more expensive than the same data in fewer, larger structures — the per-key overhead is tens of bytes, so a million tiny keys costs far more in bookkeeping than in data.

Compact encodings

Small collections are stored in a **compact serialised form** (listpack, or ziplist in older versions) rather than as full data structures — much less memory, at the cost of O(n) operations. Redis converts automatically when a threshold is crossed.

```
# see how a key is actually stored
OBJECT ENCODING mykey
  → listpack | hashtable | skiplist | intset | quicklist

# the thresholds that trigger conversion
hash-max-listpack-entries 128
hash-max-listpack-value    64
zset-max-listpack-entries 128
set-max-intset-entries    512
```

Conversion is one-way in practice. Once a collection is promoted to the full structure it does not convert back when it shrinks. A brief spike above the threshold permanently raises that key's memory cost.

Fragmentation

- mem_fragmentation_ratio — allocated to the process versus used by data
- **Above ~1.5** suggests real fragmentation; **below 1.0** means part of the dataset has been swapped to disk, which is far worse
- activedefrag yes lets Redis defragment incrementally
- **Disable swap on the host.** A swapped Redis is catastrophically slow and looks like a network problem

Investigate with --bigkeys and --memkeys from redis-cli, and MEMORY USAGE key for a single key. These are sampling tools and safe to run against production, unlike the commands they replace.

4 · ARCHITECTURE AND DEPLOYMENT TOPOLOGIES

what runs inside one server, and the three ways to deploy it — each solving a different problem

INSIDE ONE REDIS SERVER — everything below the dataset is what keeps it fast; everything beside it is what keeps it durable

Clients
application servers
connection pooled
RESP2 or RESP3
pipelining for throughput
TCP 6379

→ **I/O layer**
· Event loop, multiplexed — epoll or kqueue
· io-threads since 6.0 — socket read and write move off the main thread
· Protocol parsing and reply serialisation
Parallel here — and nowhere else

→ **Command processor**
· **Single thread.** One command at a time
· Lookup, execute, mutate the dataset
· Atomic by construction — no locks needed
· Propagates writes to replicas and AOF
A slow command here stalls everyone

→ **In-memory dataset**
· Hash table of key → value object
· Values are real data structures
· Separate table of keys with expiry
· Compact encodings for small collections
This is the whole database

Background work — the part that is genuinely threaded or forked
· BGSAVE forks a child process to write an RDB snapshot
· AOF rewrite forks similarly to compact the log
· UNLINK and lazy-free reclaim memory on a background thread
· Active expiry cycle samples keys incrementally
The fork is why a save can briefly double memory — copy-on-write

PERSISTENCE TO DISK — optional, and neither option is free
· **RDB** — a point-in-time snapshot, compact and fast to load, but everything since the last snapshot is lost on a crash
· **AOF** — an append-only log of write commands, replayed on start. Larger, slower to load, far smaller loss window
· **Both can run together, and modern Redis can write an RDB preamble inside the AOF for fast loading with fine-grained recovery**

REPLICATION IS ASYNCHRONOUS — the most important property on this diagram
A primary acknowledges a write to the client *before* the replica has it. If the primary fails in that window, the write is gone — even though the client was told it succeeded. This is not a bug or a misconfiguration; it is the design, and it is what makes Redis fast.
Consequence: never treat Redis as the system of record for anything you cannot afford to lose. Panel 7 covers the partial mitigations.

HOW A COMMAND ACTUALLY FLOWS
Client sends → I/O thread reads and parses the RESP protocol → **the single command thread executes it against the dataset** → the write is appended to the AOF buffer and queued for replicas → an I/O thread serialises and writes the reply. Replication and AOF flushing happen **after** the reply, not before — which is precisely why both durability and replication have a loss window.
Pipelining works because the client can send many commands without waiting: the round trips collapse, while execution remains strictly sequential.

THREE DEPLOYMENT TOPOLOGIES — they solve different problems, and are frequently confused

1 · STANDALONE (+ replicas)
One primary. Optional replicas serve reads and stand by.
Solves: read scaling, and a warm copy of the data.
Does not solve: automatic failover. A human promotes.
Memory ceiling is one machine. Simplest to run and reason about, and correct for the large majority of workloads.
Reads from replicas may be stale — replication is async.

2 · SENTINEL (high availability)
Separate monitoring processes (port 26379) watch the primary, agree by quorum that it has failed, elect a replica and promote it.
Solves: automatic failover, without sharding.
Does not solve: memory or write scaling — still one primary.
Run an odd number, at least three, on separate hosts.
Clients must be Sentinel-aware to find the new primary.

3 · CLUSTER (horizontal scale)
Data is partitioned across primaries by hash slot. Each primary owns a subset, and has replicas that can be promoted automatically.
Solves: memory and write scaling, plus failover.
Costs: multi-key operations are constrained — see panel 9.
Clients must be cluster-aware and follow redirections.
The most capable and the most operationally demanding.

CHOOSING BETWEEN THEM — in order, and stop at the first that fits
Does the working set fit comfortably in one machine's memory, with headroom? → **Standalone.** Add replicas if reads need scaling or a warm standby is wanted.
Do you also need failover without a human? → **Sentinel.** Does the data exceed one machine, or writes exceed one core? → **Cluster.**
Adopting Cluster before you need it buys real constraints — no cross-slot multi-key commands, limited transactions — in exchange for scale you are not using.

WHERE REDIS SITS ON CAP — and why it is not a contradiction
A single instance is not a distributed system, so CAP does not apply to it. **Redis Cluster favours availability over consistency:** because replication is asynchronous, a failover can promote a replica that is missing recently acknowledged writes, and those writes are lost. The cluster stays available; the data does not stay complete.
That trade is correct for a cache and wrong for a ledger. Which one you are building determines whether Redis belongs in the design at all.

SCALING — what each axis actually buys, and where it stops

VERTICAL — bigger machine
· More memory raises the dataset ceiling. This works and is often enough
· More cores **do not raise command throughput** — execution is one thread
· io-threads uses extra cores for network work only, and helps mainly with large values or very high connection counts
Ceiling: one core's command rate, and one machine's RAM

READ REPLICAS
· Scales **reads** linearly by adding replicas
· Does nothing for writes — every write still goes through one primary
· Reads may be stale, and the staleness is unbounded during lag
· replica-serve-stale-data decides behaviour when the link is down
Ceiling: write throughput, and replication bandwidth

HORIZONTAL — cluster or client sharding
· Scales memory **and** writes by partitioning across primaries
· Each shard is still single-threaded — you are adding threads by adding processes
· Multi-key operations only work within a slot; see hash tags in panel 9
· Resharding moves slots online, but it is real operational work
Ceiling: hot keys and skew — one popular key still lands on one shard

THE SCALING TRAP WORTH NAMING
Adding shards does not help a **hot key**. If one key receives a disproportionate share of traffic — a trending item, a global counter, a popular session — it lives on exactly one shard, and that shard's single thread is the limit no matter how many nodes you add. Partitioning distributes *keys*, not *load on a key*.
The fixes are all client-side: replicate the key across several shards with a suffix and read a random one; cache it in the application process for a short TTL; or split a counter into N sub-counters and sum them on read. None of these come from the cluster — they are design decisions you make in advance.
Diagnose it with the per-key metrics in redis-cli --hotkeys, or by sampling MONITOR briefly — never leave MONITOR running, it is itself a load source.

READING THE DIAGRAM — the four properties that explain everything else on this sheet

1 · Command execution is serial. This gives free atomicity and removes all locking — and it makes command complexity an availability concern rather than a performance one. Every "Redis went down" incident traced to an O(n) command comes from this single property.

2 · Everything is in memory. That is the speed and the constraint. Capacity planning is memory planning, eviction policy is a design decision rather than a default, and the working set not fitting is not a tuning problem but a change of architecture.

3 · Durability and replication are both asynchronous by default. The reply is sent before the AOF is flushed and before replicas have the write. Both windows are configurable and neither can be closed entirely without giving up the performance that motivated using Redis.

4 · Scaling out means more processes, not bigger ones. A shard is a single thread. Horizontal scale adds threads by adding instances — and brings the multi-key constraints that come with partitioning.

A useful way to interrogate any Redis design: what is the working set in bytes, what is the largest single key, what is the most complex command run against it, and what happens to the application if the whole dataset disappears?

If the answer to the last question is "we lose data we cannot recreate", Redis is being used as a system of record — and the asynchronous replication in property 3 means that decision needs revisiting.

5 · PERSISTENCE — RDB, AOF, AND WHAT EACH ACTUALLY GUARANTEES

MID the two mechanisms · STAFF the loss window, and the fork cost nobody plans for

	RDB — snapshot	AOF — append-only log
What it is	A point-in-time binary dump of the whole dataset	A log of every write command, replayed on start
Trigger	save rules, or BGSAVE	Continuous, flushed per appendfsync
Loss window	**Everything since the last snapshot** — potentially minutes	Down to about one second with the default
File size	Compact	Larger; needs periodic rewrite to compact
Restart speed	**Fast** — load a binary image	Slower — replay the log
Best for	Backups, replica seeding, disaster recovery	Minimising data loss

The appendfsync choice

always	fsync on every write. Safest, and a very large throughput cost. Rarely used
everysec	**The default.** A loss window of roughly one second. The sensible balance
no	The operating system decides. Fastest, and the loss window is unbounded

Run both, and let the AOF carry an RDB preamble. Modern Redis can write an RDB image at the head of the AOF file followed by the incremental commands — giving fast loading *and* a small loss window. This is the sensible default for anything you care about.

STAFF **The fork cost, which surprises people in production.** Both BGSAVE and AOF rewrite **fork the process.** The child shares memory copy-on-write, so pages modified during the save are duplicated. On a write-heavy instance this can approach **double the memory** at the moment of the save — and if the host has no headroom, the fork fails or the OOM killer intervenes.

What follows from that

- **Size the host for roughly twice the dataset**, or accept that saves may fail under load
- **Set vm.overcommit_memory = 1** on Linux, or the fork can be refused outright
- **Disable transparent huge pages** — they materially increase copy-on-write cost and latency during saves
- **Watch rdb_last_bgsave_status and aof_last_write_status.** A silently failing save means no backups, and nothing else will tell you
- **Take snapshots on a replica** where you can, keeping the fork off the primary entirely

Persistence is not a backup. The file lives on the same host as the instance that might die. Copy snapshots off the machine on a schedule, and **test a restore** — an untested restore is a hypothesis, and restore time is a number you should have measured.

Persistence does not make Redis durable in the database sense. With everysec there is a window; with replication there is a second, independent window. Both are acceptable for a cache and neither is acceptable for a ledger — which is the decision, not the configuration.

6 · EXPIRY & EVICTION

two different mechanisms

These are constantly conflated. Expiry removes a key whose TTL has passed. **Eviction** removes keys because memory is full — and it will remove keys that have not expired, and may remove keys with no TTL at all, depending on policy.

How expiry actually works

- **Lazy** — when a key is accessed, if it has expired it is removed and treated as missing
- **Active** — a background cycle samples keys with TTLs and removes expired ones incrementally
- **Consequence:** an expired key that is never accessed **still occupies memory** until the sampler happens to reach it. Expired does not mean immediately freed

The eviction policies

Policy	Removes
noeviction	**The default.** Nothing. Writes fail with an error once memory is full
allkeys-lru	Least recently used, from all keys
allkeys-lfu	Least frequently used, from all keys
allkeys-random	A random key
volatile-lru	Least recently used, **only among keys with a TTL**
volatile-lfu	Least frequently used, only among keys with a TTL
volatile-random	Random, only among keys with a TTL
volatile-ttl	The key closest to expiring

The default is the trap. noeviction means that when a cache fills, **writes start failing** rather than old entries being discarded — which is almost never what someone deploying a cache intends. Set the policy deliberately.

And the volatile-* trap. If memory fills and *no* keys have a TTL, a volatile policy has nothing eligible to evict and behaves like noeviction — writes fail while the instance looks healthy.

Choosing: allkeys-lru for a pure cache; allkeys-lfu when a stable popular subset matters more than recency; volatile-* only when the instance mixes cache data with data that must never be evicted — and set maxmemory explicitly, because an unset limit means the host's memory is the limit.

7 · REPLICATION, FAILOVER AND WHAT YOU CAN AND CANNOT GUARANTEE

STAFF the honest account of Redis's consistency guarantees — and the limits of every mitigation

How replication works

- **Asynchronous by default.** The primary replies to the client, then propagates to replicas
- **Full sync on first connect** — the primary forks, produces an RDB, ships it, then streams the backlog
- **Partial resync** after a brief disconnect, using the replication backlog and an offset, avoiding a full transfer
- **repl-backlog-size** determines how long a replica can be disconnected before a full sync is required — too small and every blip causes a fork on the primary
- **Replicas are read-only** by default and can chain from other replicas to reduce primary load

The guarantee, stated plainly: acknowledged writes can be lost. The primary confirms a write to the client before any replica has it. If the primary dies in that window and a replica is promoted, that write never existed as far as the new primary is concerned — and the client was told it succeeded.

Partial mitigations, and their limits

WAIT n ms	Blocks until *n* replicas acknowledge. Reduces the window — but it is **not a transaction**, does not roll back on timeout, and does not make the write atomic across nodes
min-replicas-to-write	The primary **refuses writes** unless enough replicas are connected. Trades availability for a smaller loss window — a genuine CP-leaning choice
AOF always	Protects against process crash, not against losing the host

Failover, and what goes wrong

- **Sentinel or Cluster detects** the primary as unreachable, agrees by quorum, and promotes a replica
- **Quorum requires an odd number**, at least three, on **separate failure domains.** Three Sentinels on one host protect against nothing
- **Clients must discover the new primary** — via Sentinel, cluster redirects, or a proxy. A client that caches the old address keeps failing after failover has succeeded
- **The promoted replica may be behind**, and the writes it lacks are gone

Split brain is real. A network partition can leave the old primary still accepting writes on the minority side while a new one is promoted on the majority side. When the partition heals the old primary is demoted and **its writes during the partition are discarded.** min-replicas-to-write limits this by making the isolated primary refuse writes.

Cross-region

- **Replicating across regions works**, and the lag is the round-trip time plus whatever the link adds
- **Do not fail over across regions automatically** on a WAN blip — the false-positive rate is high and the cost of an unnecessary failover is real
- **Active-active requires conflict resolution**, which open-source Redis does not provide; commercial and Enterprise offerings implement it with conflict-free replicated data types

The design rule that follows from all of this. Treat Redis as **rebuildable.** If losing the entire dataset means degraded performance until it repopulates, the architecture is sound. If it means lost business data, the design has a defect that no Redis configuration setting will fix.

8 · SENTINEL

automatic failover, no sharding

Sentinel is a **separate process** — not a Redis mode — that monitors primaries and replicas, decides by agreement when a primary has failed, and orchestrates promotion.

How a failover proceeds

1. Sentinels continuously ping the primary. One that stops responding is marked **subjectively down** by whichever Sentinel noticed
2. Sentinels ask each other. Once **quorum** agree, it becomes **objectively down**
3. Sentinels elect a leader among themselves to run the failover
4. The leader selects a replica — preferring one that is well-connected, has a low priority number and the most complete data
5. It promotes that replica and reconfigures the others to follow it
6. Clients are notified through pub/sub and reconnect to the new primary

Quorum and majority are different things, and this catches people out. The *quorum* is how many Sentinels must agree the primary is down. Actually *performing* the failover additionally requires a majority of all Sentinels. With three Sentinels and quorum 2, losing two of them means no failover can occur even though the quorum number is met.

Getting it right

- **Three or five Sentinels**, on separate hosts and separate failure domains from each other
- **down-after-milliseconds** — too low causes failover on transient blips; too high delays recovery
- **Set replica-priority** so an unsuitable replica — a cross-region one, or a backup target — is never promoted
- **Use a Sentinel-aware client.** Pointing at a fixed IP defeats the entire mechanism
- **Test failover deliberately.** An untested failover is the most common reason recovery does not work when it is needed

Sentinel does not shard. Memory and write throughput remain bounded by the single primary. If the constraint is capacity rather than availability, Sentinel is the wrong tool and Cluster is the answer.

9 · REDIS CLUSTER — PARTITIONING, AND THE CONSTRAINTS IT IMPOSES

MID hash slots · STAFF what stops working, and how to design around it

The model

- The keyspace is divided into **16,384 hash slots.** A key's slot is derived from a hash of its name
- Each primary **owns a subset of slots**; replicas of that primary can be promoted automatically
- There is **no proxy** — clients hold a slot map and connect directly to the right node
- A client contacting the wrong node receives a MOVED redirect and updates its map; ASK indicates a slot currently migrating
- Slots can be **moved between nodes online**, which is how resharding works

The constraint that changes application design. A multi-key command only works if every key is **in the same slot.** Otherwise the server returns a CROSSSLOT error. This affects MGET, MSET, set operations, MULTI blocks and Lua scripts — all of which work fine on standalone and fail on cluster.

Hash tags — the escape hatch

```
# only the part inside {} is hashed
user:{1234}:profile   ┐
user:{1234}:sessions  ├ same slot, same node
user:{1234}:cart      ┘ multi-key works

user:1234:profile     ┐ different slots
user:1234:cart        ┘ CROSSSLOT error
```

Use hash tags deliberately and sparingly. They are the tool for co-locating related keys — and over-using one tag concentrates a large share of the keyspace onto one node, recreating the imbalance the cluster exists to prevent.

What stops working, or behaves differently

Feature	On cluster
Multi-key commands	Same slot only
Transactions	All keys must be in one slot
Lua scripts	All accessed keys must be in one slot, and declared
Multiple databases	**Not supported** — SELECT is unavailable; database 0 only
Pub/Sub	Classic pub/sub broadcasts to all nodes, which does not scale. **Sharded pub/sub** (SSUBSCRIBE, added in Redis 7) keeps messages within a slot
KEYS / SCAN	Per node — you must iterate every node yourself

Sizing and operating

- **At least three primaries**, because failover needs a majority of primaries to agree
- **A replica per primary at minimum**, on a different failure domain
- **Replicas do not add write capacity** — only more primaries do
- **Resharding is online but not free** — it moves real data and competes with live traffic
- **Watch for slot imbalance**, and for one node holding disproportionate memory

An honest comparison worth making. Client-side sharding across independent instances gives most of the capacity benefit with far simpler operations — at the cost of manual rebalancing and no automatic failover. Cluster is worth its complexity when you need **both** horizontal scale and automatic failover; below that bar, simpler topologies usually win.

10 · TRANSACTIONS & SCRIPTING

a common misconception

Redis transactions do not roll back. MULTI queues commands and EXEC runs them as an **uninterrupted unit** — nothing interleaves. But if one command fails at runtime, for example operating on the wrong type, **the others still execute and the effect stands.** There is no rollback, and there never has been.

```
MULTI
  INCR counter          # queued
  LPUSH mylist "x"      # queued — wrong type!
  INCR counter          # queued
EXEC
# → counter incremented TWICE, LPUSH errored,
#   and nothing is undone
```

Optimistic locking with WATCH

```
WATCH balance
val = GET balance
MULTI
  SET balance val-100
EXEC        # nil if balance changed → retry
```

This is check-and-set, not a lock. If a watched key is modified by anyone between WATCH and EXEC, the transaction aborts and the client must retry. Under contention that retry loop can spin, so bound it.

Lua scripting — usually the better tool

- **A script runs atomically**, as a single command, with no interleaving
- It can **read a value and branch on it**, which MULTI cannot
- **Declare all keys** in KEYS[] — required for cluster routing
- **Keep scripts short.** A script blocks the entire server for its duration, so a loop over a large collection is an outage
- **Scripts must be deterministic**; avoid non-deterministic sources inside them
- **Functions** (Redis 7) give a managed, named alternative to EVAL with proper registration

Pipelining is not a transaction. It batches commands to remove round trips, but other clients' commands can interleave and there is no atomicity. Use pipelining for throughput, scripts for atomicity — they solve different problems and are often confused.

11 · CACHING PATTERNS

the dominant use case

Pattern	Behaviour
Cache-aside lazy loading	App reads cache; on a miss it loads from the database and populates. **The default**, and only requested data is cached
Read-through	The cache layer loads on a miss. Same effect, hidden behind an abstraction
Write-through	Write to cache and database together. Consistent, and it slows every write
Write-behind	Write to cache, flush asynchronously. Fast, and it **risks losing writes** — rarely appropriate with Redis given its own durability window

The three failure modes that cause outages

Stampede	A popular key expires and thousands of requests miss simultaneously, all hitting the database at once. **Fix:** jitter TTLs, single-flight lock on recompute, or refresh hot keys early
Penetration	Repeated requests for keys that do not exist pass straight through every time. **Fix:** cache the negative result briefly, or use a Bloom filter
Avalanche	Many keys expire at the same moment — often because they were loaded together with an identical TTL. **Fix:** randomise TTLs

Cold start is the one that is genuinely dangerous. After a restart or a flush, the cache is empty and the database receives the full read load — traffic it was never sized for, because the cache was absorbing most of it. Warm before taking traffic, and never flush an entire production cache at once.

Key design

- **A consistent, documented scheme** — app:entity:id:field
- **Include a version** in the key so a format change is a miss rather than a corrupt read
- **Always set a TTL.** Unbounded keys become an eviction incident later
- **Include every dimension the value varies on** — omitting tenant or user identity from a key serves one customer's data to another

12 · DISTRIBUTED LOCKS

and their genuine limits

The single-instance pattern

```
# acquire: atomic, unique token, always a TTL
SET lock:resource <uuid> NX PX 10000
  NX → only if it does not exist
  PX → auto-expire, so a crash cannot deadlock

# release: must be atomic, via a script
if redis.call("GET", KEYS[1]) == ARGV[1] then
  return redis.call("DEL", KEYS[1])
else return 0 end
```

The token is not optional. Without checking it, a client whose lock already expired can delete a lock now held by someone else. A plain DEL on release is a real and common bug.

The limit that matters: a lock with a TTL is not mutual exclusion. Client A acquires a 10-second lease and then stalls — a garbage-collection pause, a scheduling delay, a slow disk. The lease expires. Client B legitimately acquires it. Client A resumes, **still believing it holds the lock**, and writes. Two holders, no error, silent corruption.

What actually fixes it

- **Fencing tokens** — the lock returns a monotonically increasing number, and the **protected resource rejects any write carrying a lower token than it has already seen.** This requires support from that resource
- **Or design so it does not matter** — make the protected operation idempotent, or move the invariant into a single database transaction where it can be enforced properly

On the multi-instance algorithm. Redis documents a multi-node locking algorithm for use without a single point of failure. Its safety under timing and partition assumptions has been **publicly debated by distributed-systems practitioners**, with the core objection being the same one above: no lease-based lock is safe without fencing. Know that the debate exists, and do not present it as settled.

The practical position. Redis locks are excellent for **reducing contention** — one instance running a scheduled job, avoiding duplicated work. They are not adequate as the sole guarantee for correctness-critical exclusion. For that, use fencing tokens or a database transaction.

13 · STREAMS vs PUB/SUB

the choice people get wrong

Pub/Sub is fire-and-forget, with no persistence whatsoever. A message is delivered to whoever is subscribed *at that moment* and then discarded. A subscriber that is disconnected, restarting, or briefly slow **loses the message permanently.** There is no acknowledgement, no replay and no record that anything was missed.

	Pub/Sub	Streams
Persistence	None	Retained in the stream
Delivery	At-most-once	At-least-once, with acknowledgement
Replay	Impossible	Read from any id
Consumer groups	No — every subscriber gets everything	Yes — work is divided across consumers
Use for	Live notifications where loss is acceptable	Work queues, event logs, anything that must not be lost

Streams, in practice

```
XADD events * type "order" id "1183"
XGROUP CREATE events workers $ MKSTREAM
XREADGROUP GROUP workers c1 COUNT 10 STREAMS events >
  # … process …
XACK events workers <id>

# messages read but never acked are "pending"
XPENDING events workers
XAUTOCLAIM events workers c2 60000 0
  # reassign work from a consumer that died
```

The pending list is the mechanism that makes streams reliable. A message delivered but not acknowledged stays pending and can be claimed by another consumer. Without a process that reclaims stale pending entries, a consumer crash silently strands work — and nothing will report it.

Cap the stream. XADD ... MAXLEN ~ 100000 bounds memory; without it the stream grows without limit. The ~ makes trimming approximate and much cheaper.

When to use a real broker instead. Streams are genuinely good for moderate volumes and simple topologies. For very high throughput, long retention, or many independent consumer groups replaying history, a purpose-built log is the better tool — and Redis is not trying to be one.

14 · CONFIGURATION, SECURITY AND OBSERVABILITY

the settings that matter, why each one matters, and what to watch once it is running

Configuration that changes behaviour

Setting	Why it matters
maxmemory	**Set it explicitly.** Unset means the host's memory is the limit, and the OOM killer decides the outcome
maxmemory-policy	Defaults to noeviction — writes fail when full. Almost never what a cache wants
appendonly appendfsync	Durability and its loss window. everysec is the sensible balance
save	RDB snapshot rules. Set to empty to disable snapshots entirely
repl-backlog-size	Too small and a brief replica disconnect forces a full resync, forking the primary
min-replicas-to-write	Refuse writes without enough replicas — trades availability for a smaller loss window
timeout tcp-keepalive	Reclaim dead connections; without this, connection counts creep upward
io-threads	Network threading (6.0+). Helps with large values and high connection counts, not with command throughput
lazyfree-*	Free large objects in the background rather than blocking the command thread
activedefrag	Incremental defragmentation when the fragmentation ratio is high

Security — the defaults are not safe on a network

- **bind to specific interfaces.** Never expose Redis directly to the internet — unauthenticated Redis on a public IP is compromised within minutes, routinely
- **protected-mode yes** — a safety net when no password and no bind are configured, not a substitute for either
- **requirepass, and prefer ACLs** (Redis 6+) — per-user credentials with per-command and per-key-pattern permissions, rather than one shared password
- **TLS** for anything crossing a network you do not fully control
- **rename-command** or ACL-deny the dangerous ones — FLUSHALL, CONFIG, DEBUG, KEYS
- **Least privilege per application** — a service that only reads a cache does not need write or admin rights

The historical default is why this matters. Redis was designed for a trusted network and shipped without authentication for years. Enormous numbers of instances remain exposed, and they are scanned for continuously. Treat network exposure as the primary risk, not password strength.

Client-side settings that matter as much

- **Connection pooling**, sized against the server's connection limit rather than per-request
- **Command timeouts** — a client with no timeout waits forever on a blocked server
- **Retry policy** — retry idempotent reads; be careful with writes
- **Cluster-aware or Sentinel-aware clients**, or failover does not reach the application

What to monitor

Metric	Why
used_memory maxmemory	Headroom. Alert well before the limit, not at it
mem_fragmentation_ratio	Above ~1.5 is fragmentation; below 1.0 means swapping — far worse
evicted_keys	Rising means the working set no longer fits
keyspace_hits / misses	Hit ratio. A fall precedes almost every read incident
blocked_clients	Clients waiting on blocking commands
connected_clients rejected_connections	Connection exhaustion, which saturates before CPU does
master_repl_offset vs replica offset	**Replication lag** — the size of your data-loss window right now
rdb_last_bgsave_status aof_last_write_status	A silently failing save means no backups
instantaneous_ops_per_sec	Throughput, and its shape over time
latest_fork_usec	How long the last fork blocked the server

The tools. INFO for everything above; SLOWLOG GET for commands exceeding a threshold — the fastest route to the O(n) command causing latency; LATENCY DOCTOR for a guided summary; redis-cli --bigkeys and --hotkeys for distribution problems. **Never leave MONITOR running** — it streams every command and is itself a significant load.

15 · PRODUCTION FAILURES — SYMPTOM, CAUSE AND FIX

what actually goes wrong, in roughly the order you will meet it

Symptom	Cause	Fix
Sudden latency spike across all clients	One O(n) command — KEYS, SMEMBERS, a big HGETALL, or a long Lua script	SLOWLOG to identify it; replace with SCAN variants; ban the command via ACL
Writes rejected, instance looks healthy	maxmemory reached with noeviction, or a volatile policy with no TTLs to evict	Set an appropriate policy; ensure keys carry TTLs; add memory or shard
Periodic latency every few minutes	The fork for BGSAVE or AOF rewrite, made worse by transparent huge pages	Disable THP; snapshot on a replica; check latest_fork_usec
Memory keeps growing	Keys without TTLs, an uncapped stream or list, or fragmentation	--bigkeys; enforce TTLs; cap streams with MAXLEN; enable active defrag
Data lost after failover	Asynchronous replication — the promoted replica never received the last writes	Expected behaviour. Use min-replicas-to-write, or do not keep the only copy here
Clients fail after a successful failover	The client cached the old primary's address and is not Sentinel- or cluster-aware	Use an aware client; verify reconnection during a failover drill
One node hot, others idle	A hot key, or an over-used hash tag concentrating the keyspace	--hotkeys; replicate the key across shards; cache it in-process
Database collapses after a Redis restart	Cold cache — the database receives load the cache was absorbing	Warm before taking traffic; never flush wholesale; stagger restarts
Repeated full resyncs	repl-backlog-size too small for the disconnect duration	Increase the backlog; investigate the network instability underneath
Everything slow, no obvious cause	The host is swapping — the fragmentation ratio will be below 1.0	Disable swap; add memory; reduce the dataset
CROSSSLOT errors after moving to cluster	Multi-key commands spanning slots	Hash tags to co-locate, or restructure to single-key operations

What breaks first

#	Limit
1	**Memory.** Always the first constraint, and it arrives as either evictions or failed writes depending on policy
2	**A single slow command** blocking the one execution thread — the most common cause of a sudden incident
3	**Hot keys**, which no amount of sharding relieves
4	**Connection limits**, which saturate well before CPU
5	**Single-core command throughput** — the real ceiling, and the last one reached

Note the order. The limit people plan for — raw operations per second — is last. The ones that actually cause incidents are memory policy and command complexity, both of which are configuration and code rather than capacity.

Degradation

- **Redis down, used as a cache** — the application should degrade to the database with a circuit breaker and load shedding, not fail
- **Redis down, used for sessions** — users are logged out. Decide in advance whether that is acceptable
- **Redis down, used for rate limiting** — fail open or fail closed? **Decide deliberately and write it down**; both are defensible and the default is usually accidental

16 · PRODUCTION CHECKLIST

before an application depends on it

- ☐ **Working set sized in bytes**, with headroom, and re-measured as it grows
- ☐ **maxmemory set explicitly** — never left to the host
- ☐ **Eviction policy chosen deliberately**, not left at noeviction
- ☐ **Volatile policy only where keys actually have TTLs**
- ☐ **Every cache key has a TTL**, and TTLs are jittered
- ☐ **Key naming scheme documented**, versioned, and includes every varying dimension
- ☐ **No KEYS, FLUSHALL or MONITOR** reachable in production
- ☐ **Dangerous commands renamed or ACL-denied**
- ☐ **Large deletes use UNLINK**, not DEL
- ☐ **Big keys identified** and bounded; streams and lists capped
- ☐ **Lua scripts are short** and declare all keys
- ☐ **Team knows MULTI does not roll back**
- ☐ **Persistence chosen with the loss window stated** as a number
- ☐ **Host sized for fork overhead**; vm.overcommit_memory=1; THP disabled
- ☐ **Swap disabled** on the host
- ☐ **Snapshots copied off-host** and a restore tested and timed
- ☐ **Save failures alerted on** — a silent failure means no backups
- ☐ **Replication lag monitored** — it is the live data-loss window
- ☐ **Loss on failover accepted explicitly**, or min-replicas-to-write set
- ☐ **Odd number of Sentinels**, three or more, on separate failure domains
- ☐ **Failover tested**, including that clients reconnect
- ☐ **Clients are Sentinel- or cluster-aware**
- ☐ **Cluster constraints understood** — CROSSSLOT, no SELECT, sharded pub/sub
- ☐ **Hash tags used sparingly**, with slot balance monitored
- ☐ **Hot-key strategy exists** before the first trending event
- ☐ **Connection pooling sized** against the server limit
- ☐ **Client command timeouts set** — never unbounded
- ☐ **Never bound to a public interface**; TLS where the network is untrusted
- ☐ **ACLs per application**, least privilege, no shared password
- ☐ **Cache stampede, penetration and avalanche** each mitigated
- ☐ **Cold-start plan exists** — warm before taking traffic
- ☐ **Locks use a token and a TTL**; released by script, never plain DEL
- ☐ **Correctness-critical exclusion does not rely on a Redis lock alone**
- ☐ **Pub/Sub not used where loss is unacceptable** — use Streams
- ☐ **Stream pending entries reclaimed** by a stale-consumer process
- ☐ **Behaviour defined for Redis being down**, per use case
- ☐ **Rate-limiter fail-open vs fail-closed decided** and documented
- ☐ **SLOWLOG reviewed regularly**, not only during incidents
- ☐ **Version and licence position understood** for your deployment

The one that summarises the sheet: **state the working set in bytes, the largest single key, the most expensive command any client can issue, and what the application does if the entire dataset vanishes.** Those four answers determine whether Redis is being used as a cache — which it is superb at — or as a database, which its asynchronous replication and loss windows mean it is not.